Datawhale

动手学大模型应用开发 · 第 6 节

Harness 与持续任务

让 Agent 在中断后仍能安全继续



主讲：陈思州

本节学习路线

Datawhale

总览主线 • 先组合前五节，再解释长任务怎样可靠运行

Harness 的价值来自部件之间的协作方式，也来自模型循环之外的运行责任。

01 课程总览

模型、循环、证据、上下文和能力

02 运行循环

构建输入、选择动作、执行并观察

03 持久恢复

状态、检查点、产物与副作用

04 治理验收

预算、权限、追踪、停止与人工介入

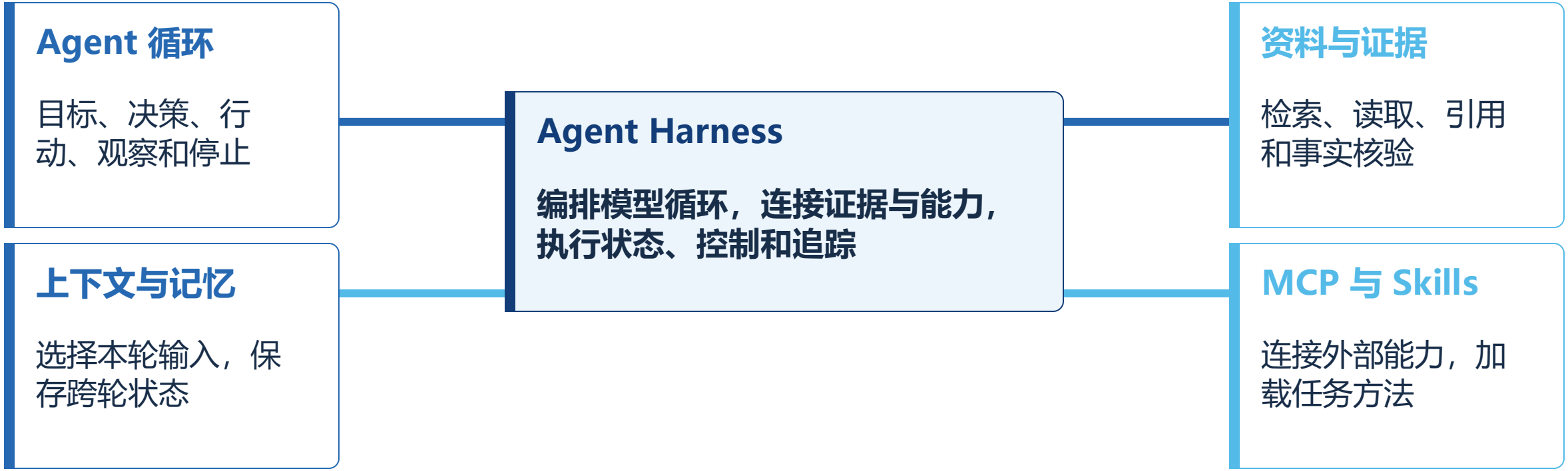
Harness 把已经学过的能力组织成可控制、可恢复、可评估的运行系统。

Harness 把前五节能力组织成可运行系统

Datawhale

课程总览 • 前五节从知识点变成一个运行中的系统

每个部件解决一种问题，Harness 决定它们何时参与、怎样交接以及失败后如何继续。



Harness 不替代这些能力，而是把它们组织成可靠运行。

一次任务会穿过整套 Harness

Datawhale

一次运行 · 每一轮都同时处理信息、能力和控制

模型只负责生成候选行动，运行系统负责让行动在正确边界内发生并留下证据。

01	**目标与状态**	任务阶段、预算、权限和未决动作
02	**证据与上下文**	检索资料，读取记忆，排列本轮 Prompt
03	**模型决策**	生成最终回答或候选工具调用
04	**能力执行**	校验参数与授权，运行工具或外部作业
05	**保存与验收**	记录结果、更新检查点并判断是否继续

Harness 的核心产物不是一段回答，而是一条可继续、可审计的运行链。

Harness 是围绕 Agent 循环的运行机制

Datawhale

定义与边界 • 模型做决策，运行机制管过程

持续任务要求状态和控制不依赖某一次模型调用或某一个进程。



Harness 的价值是把智能选择放进可持久、可约束、可检查的运行环境。

Harness 把提示工程扩展为循环工程

Datawhale

A

提示工程

优化一次模型请求中的指令、示例、上下文结构和输出格式，目标是提高单轮决策质量。

边界

B

循环工程

设计多轮状态、工具执行、预算、停止条件、恢复路径和验收证据，目标是让任务持续可控。

运行责任要覆盖整条控制路径

Datawhale

控制路径 · 每一步都要有输入、边界和持久结果

恢复能力不是最后补一个“继续”按钮，而是整条执行路径的设计结果。

1. **接收任务**

   保存目标、验收、权限与来源

2. **构建上下文**

   从持久状态选择本轮信息

3. **选择执行**

   模型提议，程序校验后调用

4. **保存结果**

   观察、产物、预算与阶段落盘

5. **验收恢复**

   根据已知状态继续、停止或人工介入

任何一段没有记录边界或结果，都可能在中断后变成不可判定窗口。

控制平面与执行平面分担不同责任

Datawhale

运行架构 • 决策与执行通过状态和回执闭环

把调度和动作混在一个进程中，会让恢复程序难以判断上一步是否真正发生。

01 控制平面

选择下一步，分配预算，设置停止和恢复策略

02 执行平面

运行模型、工具或外部作业，不自行改变任务目标

版本化运行状态

任务阶段、授权、预算、尝试 ID 和最后一次确认结果

03 命令

携带动作参数、授权范围、修订号与幂等键

04 回执

返回原始结果、副作用状态、错误和证据位置

控制平面决定能做什么，执行平面证明实际做了什么。

持续任务需要四类持久对象

Datawhale

持久模型 · 不用一个 JSON 同时充当状态、大文件和日志

这四类对象共同回答：任务现在在哪里，已经产生什么，又是怎样走到这里。

持续任务 跨进程保留进度、结果与可追溯历史

01 任务状态

目标、阶段、预算、待处理动作

02 检查点

可恢复的一致快照与修订号

03 产物

源码、报告、工具结果与内容哈希

04 事件轨迹

开始、完成、失败、恢复与操作来源

状态支持推进，检查点支持恢复，产物保留内容，事件支持诊断。

任务、运行与尝试不应混成一个编号

Datawhale

标识结构 • 同一任务可以有多次运行和多次动作尝试

把编号分层，才能在重试、恢复和审计时准确说明哪一次发生了什么。

01	**任务**	长期目标与成功条件
02	**一次运行**	某个进程的启动到停止
03	**动作尝试**	一次模型或工具调用
04	**外部结果**	绑定尝试 ID 与幂等键

任务 ID 保持连续，运行 ID 区分进程，尝试 ID 关联具体副作用。

快照与事件记录提供两种恢复视角

Datawhale

A

状态快照

保存某个一致时刻的当前状态，恢复读取快，但无法单独解释状态如何演变。

边界

B

事件记录

按顺序保存状态变化和外部回执，可以重建过程，但需要处理版本、顺序与压缩。

任务阶段必须显式、可验证

Datawhale

状态机 • 每个阶段都要有允许的输入、转移和恢复策略

单一 running 字段无法区分模型请求在途、工具待执行和结果待保存。

01	**就绪**	状态一致，可构建下一轮输入
02	**决策中**	模型请求已预留预算
03	**动作待执行**	工具名、参数和尝试 ID 已保存
04	**结果待保存**	外部动作可能已发生
05	**停止分流**	完成、失败或进入人工审核

恢复时先读阶段，再决定继续、重试、核对还是停止。

检查点要放在可解释的边界

Datawhale

检查点时序 · 先记录将要做什么，再记录实际发生了什么

恢复不是回到任意一行代码，而是回到一个状态与外部环境关系可以说明的边界。

保存意图

动作、参数、尝试 ID 与幂等键

1

2

执行动作

调用外部系统或本地工具

保存结果

原始返回、错误、来源与时间

3

4

更新阶段

已知结果后才标记可继续

下一轮

从新状态构建上下文

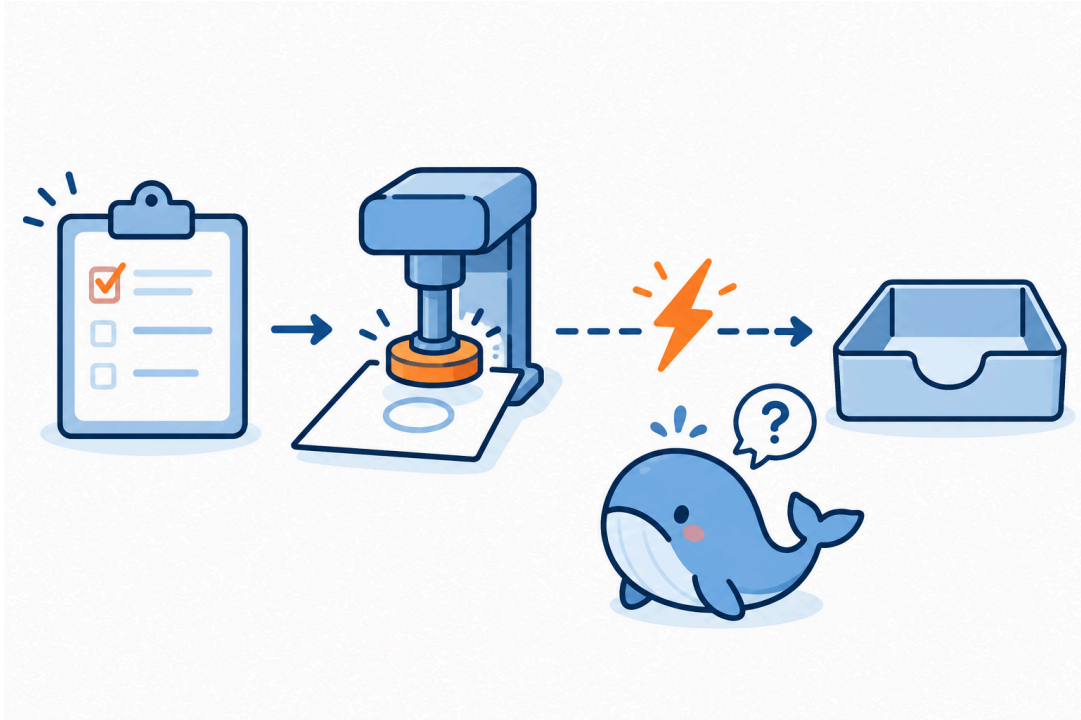
5

检查点要保存“已知事实”，不能把未知结果提前写成成功。

故障发生在副作用前后，恢复策略不同

Datawhale

工具可能已经执行，但检查点尚未保存结果，这是恢复最难的一刻。



调用已发起，结果却还没有保存

中断发生在哪里

记录意图、调用工具、保存结果。
在后两步之间退出，恢复者
只能看到 tool_inflight。

能不能直接重试

只读动作可评估重新读取；
“提交订单”却可能被重复执行。
需按业务选择幂等键、状态查询或人工处理。

本课只读实验不能证明任意写操作都能安全重试。检查点也不自动保证恰好执行一次。

租约与修订号阻止两个执行器同时推进

Datawhale

并发控制 • 每次只允许一个执行器拥有推进权

进程重启和重复投递会产生竞争，单靠任务 ID 无法阻止旧执行器继续写入。

1. **读取修订**
   获得任务当前阶段、revision 和未决动作
2. **取得租约**
   在限定时间内声明本执行器拥有推进权
3. **执行一步**
   动作携带尝试 ID、授权和当前修订
4. **提交新修订**
   只接受基于最新 revision 的状态更新
5. **释放或续租**
   完成后释放，长动作需在边界前安全续租

租约处理谁在运行，修订号处理谁可以写入。

恢复任务必须继承原有运行边界

Datawhale

恢复前置检查 · 重启不代表新任务

同一任务的预算、权限和输入来源必须跨运行延续，否则可以通过重启绕过控制。

01	读取授权	用户、范围、工具与人工确认
02	读取预算	模型、工具、时间和费用累计值
03	核对版本	模型、工具、资料与 Skill 是否变化
04	构建输入	只把本轮需要的状态装入上下文

恢复是继续受约束的任务，不是开启一次无历史的新循环。

幂等键能阻止重复生效，不能证明结果正确

Datawhale

恢复前已用掉的资源必须继续累计，权限也要再次检查。

怎样理解

把模型预算设为 1，执行后保存。
重新打开任务，观察剩余预算。
再让模型提出未注册的写工具。

具体例子

预算耗尽：停止并保存明确原因。
未注册写动作：执行前拒绝。
重启应用不应把预算计数恢复成 0。

允许范围由运行时落实。提示词中写了“可以”并不构成程序授权。

护栏要覆盖输入、决策、执行与输出

Datawhale

安全控制 · 沿整条运行路径设置检查点

只审查最终文本会遗漏工具参数、外部写入和已经发生的副作用。

输入边界

检查来源、敏感数据、任务范围和提示注入风险

1

2

决策边界

检查工具选择、参数、授权范围和剩余预算

3

执行边界

限制文件、网络、进程、并发和可回滚操作

4

输出与副作用

核验回答、产物、外部状态变化和人工确认要求

护栏与日志共同覆盖决策前后，才能在失败时定位并恢复。

补偿动作与事务回滚不是同一种保证

Datawhale

A

事务回滚

同一受控事务内撤销尚未提交的变化，目标是恢复到提交前的一致状态。

边界

B

业务补偿

副作用已经发生后执行新的反向动作，例如撤销预约或发送更正，原事件仍然保留。

检查点、产物、事件与日志承担不同证据职责

Datawhale

证据分层 • 恢复、复用、审计和诊断需要不同数据

不要把所有信息都塞进一个日志字符串，也不要把日志可用性变成任务正确性的前提。

可追溯运行　从任务状态找到结果，再沿事件和日志说明过程

01 检查点

任务阶段、修订号与恢复依据

02 产物

可读内容、来源、大小与内容哈希

03 事件

结构化的开始、结果、失败与转移

04 日志

排错上下文、耗时与运行环境细节

检查点让任务继续，产物让结果可用，事件与日志让故障可定位。

故障诊断要从停止阶段逐层追到原始证据

Datawhale

诊断路径 · 先确定任务事实，再展开更细的运行细节

证据链的价值是将“又跑一次”替换为“定位失败层后采取对应动作”。

01	**停止阶段**	当前状态、停止原因与累计预算
02	**事件轨迹**	最后已知动作、结果与转移
03	**原始产物**	内容、来源、哈希与关联尝试
04	**运行日志**	环境、耗时、异常与底层细节
05	**处置策略**	重试、补偿、人工修复或停止

先区分状态失败、工具失败、环境失败和结果失败，再选择恢复动作。

恢复流程要从已知状态重建下一步

Datawhale

恢复顺序 • 恢复的对象是任务状态，不是一个无限增长的对话文本

每次只恢复到一个可解释边界，完成一步并保存，再决定是否继续。

1. **读检查点**

   任务阶段、预算、授权与待处理动作

2. **校验版本**

   修订号、模型、工具、资料与 Skill

3. **处理未决**

   查询外部状态或进入人工审核

4. **重建输入**

   选取当前目标、已知观察与必要产物

5. **继续一步**

   执行、落盘，然后再次检查状态

恢复是受校验的状态转移，不是把所有历史重新喂给模型。

设计 Harness 时先回答五个问题

Datawhale

设计检查 · 先明确持久语义与副作用边界，再追求更长自动运行

每个问题都应有可在数据结构、事件或产物中核对的答案。

01	状态	最小恢复快照与独立产物是什么
02	落盘	外部动作前后分别保存什么
03	重试	哪些动作幂等，哪些需要审核
04	边界	预算、权限和版本如何跨运行延续
05	证据	能否从结论追到原始结果与尝试

可恢复只是起点；可解释的副作用、延续的边界和可追溯证据才能支撑持续任务。

停止条件需要区分完成、暂停与放弃

Datawhale

终止语义 · 停下来也必须说明系统处于什么状态

只用 running 和 done 两个状态，会把等待、阻塞、失败和真正完成混在一起。

可解释停止　　停止原因、证据、后续责任和可恢复入口同时记录

01 完成

成功条件全部通过，产物和副作用已核验

02 等待

保存外部事件条件，唤醒前不继续调用模型

03 人工确认

展示拟执行动作、风险和当前证据，由人决定

04 终止

预算、策略或风险触发停止，保留未完成原因

停止状态决定任务能否继续、由谁继续以及从哪里继续。

本节知识结构

Datawhale

1. **前五节的模型、证据、上下文和能力在 Harness 中形成一次完整运行。**
2. Harness 管理循环、状态、预算、权限、追踪、人工介入与停止。
3. 持续任务还要处理并发、未决副作用、补偿和跨进程恢复。
4. 下一节用 Task、Trial、Trace、Outcome 和 Grader 检查整套系统。



参考文献与延伸阅读

Datawhale

本节关于 Agent Harness、运行循环、持久执行与人工介入主要参考以下资料。

[1] Anthropic — **Effective Harnesses for Long-Running Agents**
anthropic.com/engineering/effective-harnesses-for-long-running-agents

[2] Anthropic — **Scaling Managed Agents: Decoupling the Brain from the Hands**
anthropic.com/engineering/managed-agents

[3] OpenAI — **Agents SDK: Running Agents and Human-in-the-loop**
openai.github.io/openai-agents-python

[4] Temporal — **Durable Execution Technical Guide**
assets.temporal.io/durable-execution.pdf

[5] Anthropic — **Demystifying Evals for AI Agents**
anthropic.com/engineering/demystifying-evals-for-ai-agents